

AURORA BACHELOR — Futuristic Bachelor House Management System

You are an expert **Full-Stack Software Architect, UI/UX Designer, Database Engineer, and Frontend Developer.**

I want you to build a production-quality web application called:

AURORA BACHELOR

Tagline

"Your House. Your Meals. Your Money. One Intelligent System."

Aurora Bachelor is a futuristic **Bachelor Mess / Shared House Management System** designed for people who live together and need to manage:

-  Daily meals
-  Members
-  Member payments
-  Monthly expenses
-  House rent
-  Electricity bills
-  Water bills
-  Internet bills
-  Grocery expenses
-  Monthly meal rates
-  Individual member balances
-  Income vs expense
-  End-of-month settlement
-  Reports
-  Data export/import

The application should eliminate the need for manually maintaining Excel sheets.

1. CORE PRODUCT VISION

The app should work like a combination of:

- Excel
- Personal finance tracker
- Meal management system
- Shared expense calculator
- Bachelor house management dashboard

But instead of looking like Excel, it should feel like a **premium futuristic financial/management system**.

The user should be able to open the app and immediately understand:

Who ate how many meals? Who paid how much? Who owes money? How much is the current meal rate? How much did the house spend? How much rent is due? What is everyone's final balance?

The application must automatically calculate everything.

2. BRAND / VISUAL IDENTITY

The name is:

AURORA BACHELOR

The visual identity should be inspired by:

- Deep Space
- Galaxy
- Aurora
- Cyberpunk
- Holographic dashboards
- Neon interfaces
- Cosmic data visualization
- Futuristic spacecraft control panels

However, DO NOT make it look childish or like a gaming website.

It should feel like a serious:

"Next-Generation House Financial Management OS."

3. UI DESIGN DIRECTION

Create a premium futuristic interface.

Background

Use:

- Deep black / dark navy space background
- Subtle galaxy particles
- Aurora gradients
- Nebula-like glow

- Very subtle stars
- Glassmorphism panels

Avoid excessive animations that hurt usability.

Visual language

Use:

- Glass cards
- Soft borders
- Neon glow
- Gradient highlights
- Holographic elements
- Smooth transitions
- Rounded modern components
- Minimal icons
- Data visualization

Color concept

Primary:

- Aurora cyan
- Electric blue
- Violet
- Magenta

Supporting:

- Green for positive balance
- Red for debt/negative balance
- Amber for warnings

Do not overuse colors.

The interface must remain readable.

4. RESPONSIVE DESIGN

The application MUST work perfectly on:

- Desktop
- Laptop
- Tablet
- Android phones
- iPhone

Mobile experience is extremely important.

On mobile:

- Bottom navigation can be used
- Cards should stack
- Tables should become responsive
- Important actions should remain easily accessible
- Daily meal entry should require minimal taps

5. APPLICATION STRUCTURE

Create the following major sections:

Dashboard

Members

Daily Meals

Expenses

Payments

Monthly Report

House Bills

Settlement

Settings

6. DASHBOARD

The dashboard should be the central command center.

Show:

Current Month

Example:

August 2026

Key statistics

Create futuristic metric cards:

Total Members 8

Total Meals 186

Meal Rate ₺62.50

Total Expenses ₺38,450

Total Collected ₺35,000

Outstanding ₺3,450

Member Balance Overview

Display something like:

Member	Meals	Paid	Meal Cost	Other Cost	Balance
Masum	28	₺2500	₺1750	₺900	-₺150
Rahim	25	₺3000	₺1562	₺900	+₺538

Use clear visual indicators.

7. MEMBER MANAGEMENT

Create a complete member management system.

Each member should have:

- Name
- Profile image/avatar
- Phone (optional)
- Joining date
- Status
- Monthly meal count
- Total payments
- Personal expenses
- Balance

Actions:

- Add member
- Edit member

- Remove member
 - View member details
-

8. DAILY MEAL SYSTEM

This is the heart of the application.

Every day should have a meal entry.

Example:

August 10

Member	Breakfast	Lunch	Dinner	Total
Masum	1	1	1	3
Rahim	0	1	1	2
Karim	1	1	0	2

Allow:

- 0 meals
- 1 meal
- Multiple meals

Provide quick buttons:

B / L / D

for:

Breakfast / Lunch / Dinner.

Users should be able to update meals quickly.

9. MEAL CALCULATION ENGINE

Implement automatic monthly meal calculation.

Formula:

Total Meals

Total Meals = Sum of all members' meals

Meal Rate

Meal Rate = Total Meal-Related Expenses / Total Meals

Individual Meal Cost

Individual Meal Cost =
Member Total Meals × Meal Rate

Do NOT hardcode calculations.

Create a centralized calculation engine.

10. EXPENSE MANAGEMENT

Users should be able to record any expense.

Expense fields:

- Date
- Category
- Description
- Amount
- Paid by
- Payment method
- Notes

Categories:

- Grocery
- Rice
- Vegetables
- Meat
- Fish
- Gas
- Electricity
- Water
- Internet
- Cleaning
- Rent
- Maintenance
- Other

Example:

Date: 10 Aug 2026
Category: Grocery
Description: Monthly বাজার
Amount: ₳4,500
Paid By: Masum

11. PAYMENT TRACKING

Members can make payments toward the house.

Example:

Masum
Payment: ₳3,000
Date: 5 Aug
Method: Cash

Store every transaction.

Show:

- Total payment
- Payment history
- Current balance
- Last payment

12. HOUSE RENT

Create a dedicated rent management section.

Track:

- Monthly rent
- Paid amount
- Remaining amount
- Who paid
- Due date

Example:

Monthly Rent: ₺20,000

Masum	₺2,500
Rahim	₺2,500
Karim	₺2,500
...	

Support equal and custom rent distribution.

13. HOUSE BILLS

Create separate bill tracking:

Electricity

Water

Gas

Internet

Cleaning

Maintenance

Each bill should support:

- Amount
- Date
- Due date
- Paid by
- Distribution method

Distribution methods:

1. Equal split
 2. Per member
 3. Custom split
-

14. MONTHLY ACCOUNTING ENGINE

At the end of the month the application should automatically calculate:

Total Expenses

Food
+ Rent
+ Electricity
+ Water
+ Gas
+ Internet
+ Cleaning
+ Maintenance
+ Other

Total Payments

Sum all member payments.

Individual Cost

For each member calculate:

Meal Cost
+
Rent Share
+
Utility Share
+
Other Shared Expenses
+
Personal Expenses

15. FINAL SETTLEMENT ENGINE

Create a dedicated:



SETTLEMENT CENTER

At the end of each month show:

Who owes?

Example:

Rahim owes ₺850
Karim owes ₺1,250

Who should receive?

Masum should receive ₺2,100

Then generate a simple settlement plan.

Example:

Rahim → Masum : ₺850
Karim → Masum : ₺1,250

The goal is to minimize unnecessary transactions.

16. MONTHLY REPORT

Create a beautiful monthly report.

Include:

- Total meals
- Total expenses
- Meal rate
- Total payments
- Total rent
- Total utility bills
- Grocery spending
- Member-wise breakdown
- Outstanding balances
- Settlement summary

Use charts:

- Expense distribution
 - Monthly spending
 - Member meal comparison
 - Payment comparison
 - Category spending
-

17. MEMBER PROFILE PAGE

Clicking a member should open a detailed profile.

Show:

Member Information

Current Balance

Total Meals

Meal Cost

Total Paid

Rent Share

Utility Share

Other Expenses

Payment History

Meal History

Monthly History

Provide a timeline-style transaction view.

18. SMART MONTH SYSTEM

Users should be able to switch months.

Example:

July 2026
August 2026
September 2026

Each month must maintain its own accounting data.

Do NOT mix data between months.

19. DATA MODEL

Design a proper relational database.

Suggested entities:

```
users
houses
members
months
meals
expenses
payments
bills
rent_records
settlements
categories
```

Create proper relationships.

Example:

```
House
├── Members
├── Months
│   ├── Meals
│   ├── Expenses
│   ├── Payments
│   ├── Bills
│   └── Settlement
```

Use database constraints where appropriate.

20. DATA PERSISTENCE

The application **MUST NOT** lose data after refresh.

If building an MVP without backend initially:

Use:

```
IndexedDB
```

instead of relying only on localStorage.

Structure the application so a backend can be added later.

If a backend is appropriate for the selected stack, implement proper persistent storage.

21. BACKUP SYSTEM

Create:

Export Data

Allow users to export:

JSON
CSV
Excel

Import Data

Allow restoring previous data.

Also create:

Backup Now

and

Restore Backup

features.

22. EXCEL COMPATIBILITY

Since users normally use Excel for this work, Aurora Bachelor should provide:

Export Monthly Report → Excel

The generated spreadsheet should contain:

- Member list
- Daily meal sheet
- Payment sheet
- Expense sheet
- Monthly summary
- Settlement sheet

23. SEARCH AND FILTER

Add global search.

Users should be able to search:

- Member
- Expense
- Payment
- Date
- Category

Add filters:

Today
This Week
This Month
Custom Date

24. ACTIVITY TIMELINE

Create a timeline:

10:30 PM
Masum added ₺2,000 payment

8:40 PM
Grocery expense ₺1,250 added

7:20 PM
Rahim's dinner marked

5:00 PM
Electricity bill updated

This creates an audit trail.

25. NOTIFICATION SYSTEM

Create smart notifications.

Examples:

⚠ Electricity bill is due tomorrow.

💰 Rahim has an outstanding balance of ₺850.

🍲 August meal rate increased to ₺64.20.

🏠 Monthly rent is not fully paid.

26. DARK / LIGHT MODE

Default:

Dark Galaxy Mode

Optional:

Light Mode

But Dark Galaxy Mode should be the primary identity.

27. AURORA UI COMPONENTS

Create reusable components such as:

```
AuroraCard  
AuroraButton  
AuroraModal  
AuroraTable  
AuroraChart  
AuroraBadge  
AuroraInput  
AuroraSelect  
AuroraDatePicker  
AuroraSidebar  
AuroraNavbar  
AuroraMetricCard  
AuroraTransactionItem
```

Maintain a consistent design system.

28. ANIMATION

Use subtle futuristic animations.

Examples:

- Aurora background movement
- Card hover glow
- Number counting animation
- Smooth page transitions
- Chart animations
- Modal transitions
- Button feedback

Avoid:

- excessive particle effects
- distracting animations
- performance-heavy effects

Performance is more important than visual effects.

29. OPTIONAL AI FEATURE

Design the architecture so an AI assistant can be added later.

Possible feature:



Aurora AI

Users could ask:

"Who owes money this month?"

"How much did we spend on groceries?"

"What is the current meal rate?"

"How much did Masum pay?"

"Why is this month's expense higher?"

"Who has the highest meal count?"

For the first version, this can be a future module.

Do not make the application dependent on an AI API.

30. PWA

Make Aurora Bachelor installable as a Progressive Web App.

Requirements:

- manifest.json
- service worker
- offline-friendly architecture
- install prompt
- app icons
- mobile-friendly interface

The app should feel like a native application when installed.

31. SECURITY

Implement basic security architecture.

Important principles:

- Validate all inputs
 - Prevent negative invalid values
 - Prevent duplicate records where inappropriate
 - Sanitize user input
 - Never expose sensitive configuration
 - Keep business logic separate from UI
-

32. ARCHITECTURE

Use a clean architecture.

Separate:

```
UI
Components
State Management
Business Logic
Calculation Engine
Data Layer
```

Do NOT put all logic into one giant JavaScript file.

33. CALCULATION ENGINE

Create a dedicated module.

For example:

```
calculateTotalMeals()  
calculateMealRate()  
calculateMemberMealCost()  
calculateHouseExpenses()  
calculateMemberShare()  
calculateMemberBalance()  
calculateMonthlySummary()  
calculateSettlement()
```

All financial calculations should come from these functions.

34. VALIDATION

The system must handle:

- Empty member
- Invalid amount
- Duplicate payment
- Negative amount
- Missing date
- Deleted member with existing transactions
- Month closing
- Editing previous meal records

Show useful error messages.

Never silently fail.

35. MONTH CLOSING

Create:



CLOSE MONTH

Before closing:

Show confirmation:

August 2026

Total Members: 8

Total Meals: 186

Meal Rate: ₹62.50

Total Expense: ₹38,450

Outstanding: ₹3,450

After closing:

- Lock historical records
- Generate final report
- Preserve data
- Allow reopening only through confirmation

36. DEMO DATA

Create realistic demo data so the application looks alive immediately.

Example members:

Masum

Rahim

Karim

Sakib

Hasan

Generate several days of meal records, expenses and payments.

37. EMPTY STATES

Do not show blank screens.

Example:

Instead of:

No data

show:



Your kitchen is quiet...

No meals have been recorded today.

[+ Add Today's Meals]

Keep the experience friendly and futuristic.

38. ERROR STATES

Design beautiful error states.

Example:



SYSTEM INTERRUPTED

Something went wrong while loading your house data.

[Try Again]

But errors should still be understandable.

39. PERFORMANCE

The app should remain fast even with:

- 50 members
- 365 days of meal records
- Thousands of transactions

Avoid unnecessary re-renders.

Use efficient state updates.

40. TECH STACK

Choose a modern stack suitable for this application.

Preferred:

Frontend

React + TypeScript

Styling

Tailwind CSS

Icons

Lucide Icons

Charts

Recharts or another lightweight chart library

State

Zustand or Context where appropriate

Data

IndexedDB for local-first MVP

Structure the project so backend integration can later use:

- PostgreSQL
- Supabase
- Firebase
- Node.js API

Do NOT introduce unnecessary technologies.

41. PROJECT STRUCTURE

Use a professional structure similar to:

```
aurora-bachelor/  
|  
├─ src/  
|   ├─ components/  
|   ├─ pages/  
|   ├─ layouts/  
|   ├─ hooks/  
|   ├─ services/  
|   └─ database/
```

```
|   |   | calculations/
|   |   | store/
|   |   | utils/
|   |   | types/
|   |   | assets/
|
|   | public/
|   |   | icons/
|   |   | manifest.json
|
|   | tests/
|
|   | package.json
|   | README.md
```

Adapt this structure if the chosen stack requires something better.

42. DEVELOPMENT PROCESS

Do NOT immediately generate a huge amount of code.

Follow this sequence:

Phase 1 — Planning

Analyze the complete product.

Define:

- Architecture
- Database schema
- User flow
- Component architecture
- Calculation logic
- Page structure

Then explain the plan briefly.

Phase 2 — Foundation

Build:

- Project setup
- Theme
- Layout
- Navigation

- Design system
 - Database layer
 - State management
-

Phase 3 — Core Features

Implement:

1. Members
 2. Daily Meals
 3. Expenses
 4. Payments
 5. Bills
 6. Rent
 7. Monthly calculations
 8. Settlement
-

Phase 4 — Analytics

Implement:

- Dashboard
 - Charts
 - Reports
 - Member analytics
-

Phase 5 — Advanced Features

Implement:

- Export
 - Import
 - Backup
 - PWA
 - Notifications
 - Month closing
-

Phase 6 — Testing

Test:

- Meal calculations
- Meal rate
- Payments

- Expenses
 - Member balances
 - Settlement
 - Month switching
 - Data persistence
 - Import/export
 - Responsive layout
-

43. IMPORTANT ACCOUNTING RULE

This is extremely important:

NEVER hardcode financial results.

Every displayed number must be derived from stored transaction data.

For example:

```
Total Expense
=
SUM(all valid expenses)
```

and

```
Member Balance
=
Total Paid
-
Total Member Responsibility
```

Make the formulas transparent and testable.

44. USER EXPERIENCE GOAL

The entire app should answer these questions within seconds:

"আজকে কে কয়টা **meal** খেল?"

"এই মাসে **meal rate** কত?"

"কে কত টাকা দিয়েছে?"

"কে কত টাকা বাকি?"

"বাসার মোট খরচ কত?"

"ভাড়া কত?"

"বিদ্যুৎ বিল কত?"

"মাস শেষে কে কাকে টাকা দেবে?"

The user should never need to open Excel to calculate these manually.

45. FUTURE ROADMAP

Architect the system so future versions can add:

Aurora Bachelor v2

- Multi-house support
- Cloud sync
- User accounts
- House invite system
- Role-based access
- Admin/member roles

Aurora Bachelor v3

- Aurora AI
- Smart expense prediction
- Expense anomaly detection
- Monthly spending prediction
- Grocery planning
- Smart shopping list

Aurora Bachelor v4

- Android app
 - iOS app
 - Real-time synchronization
 - Push notifications
 - Multi-language support
 - Bengali + English
-

46. FINAL DESIGN PHILOSOPHY

The final product should NOT feel like:

✗ Excel replacement

It should feel like:



"A Personal Operating System for Bachelor House Life."

Every interaction should be:

Simple → Fast → Automatic → Visual → Trustworthy

The user should enter data once.

Aurora Bachelor should do the mathematics automatically.

47. CLAUDE CODE EXECUTION RULES

You are working inside a real development environment.

Before modifying files:

1. Inspect the existing project.
2. Understand the current architecture.
3. Do not overwrite useful existing code unnecessarily.
4. Reuse components where appropriate.
5. Keep code modular.
6. Use TypeScript types properly.
7. Avoid duplicated business logic.
8. Write reusable functions.
9. Test important calculations.
10. Run the application after major changes.
11. Fix errors before moving forward.

When you encounter an architectural decision, choose the solution that is:

simple + scalable + maintainable + production-ready.

Do not add dependencies just because they are popular.

48. FIRST TASK

For now, DO NOT build the entire application at once.

Start with:

STEP 1

Inspect the current repository/project.

STEP 2

Create a complete architecture plan.

STEP 3

Define the database/data model.

STEP 4

Define the calculation engine.

STEP 5

Define the main UI structure.

STEP 6

Create the initial futuristic Aurora Bachelor dashboard shell.

After completing these steps, show me:

1. Project architecture
2. Database schema
3. Calculation formulas
4. Component structure
5. Files created/modified
6. What has been implemented
7. What should be built next

Then wait for my instruction before moving to the next major feature.

END OF SPECIFICATION

Build Aurora Bachelor as a serious, polished, futuristic product — not as a simple CRUD demo.